

Bases de Dados Relacionais e SQL I

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Universidade de Aveiro

Introdução

O que é uma Base de Dados?

Um conjunto estruturado de dados mantido num computador, especialmente um que seja acessível de várias formas.

- **Objetivo:** Armazenar, recuperar e gerir grandes quantidades de informação.
- **Evolução:**
 - **Flat files:** CSV/Texto (Rápido mas limitado).
 - **Hierárquico:** Estrutura em árvore (IBM IMS).
 - **Relacional:** Tabelas com relações (O padrão desde os anos 80).
 - **NoSQL:** Esquemas flexíveis para big data (MongoDB, Redis).

Porquê não usar apenas Ficheiros? I

- **Redundância:** Os dados são frequentemente duplicados em vários ficheiros.
- **Inconsistência:** Se alterar a morada de um utilizador num ficheiro, pode esquecer-se dos outros.
- **Concorrência:** O que acontece se duas pessoas tentarem escrever no mesmo ficheiro ao mesmo tempo?
- **Segurança:** Difícil de controlar quem vê cada parte dos dados.

Porquê não usar apenas Ficheiros? II

- **Escalabilidade:** Pesquisar em 1 milhão de linhas num CSV é lento ($O(N)$).
- **Relações:** Como ligar um aluno em `alunos.csv` a uma nota em `notas.csv` de forma eficiente?
- **Integridade de Dados:** Como garantir que a “Idade” é sempre um número e não “Vinte”?

O Modelo Relacional

Proposto por Edgar F. Codd em 1970, organiza os dados em uma ou mais tabelas.

- **Tabela (Relação):** Uma coleção de elementos de dados organizados em linhas e colunas.
- **Linha (Tuplo/Registro):** Representa um item único na tabela.
- **Coluna (Atributo/Campo):** Representa uma propriedade específica dos itens.

- **Chave Primária (PK):** Um identificador único para cada linha.
 - **Chave Natural:** Algo que já existe (ex: NIF).
 - **Chave Substituta:** Um ID artificial (ex: `id = 1, 2, 3`).
- **Chave Estrangeira (FK):** Uma coluna que cria uma ligação entre duas tabelas.
- **Chave Composta:** Uma chave primária composta por duas ou mais colunas.

Garante que as relações entre tabelas permanecem consistentes.

- Se `alunos.dept_id` referencia `departamentos.id`, a base de dados impedirá:
 - Apagar um departamento que ainda tenha alunos.
 - Adicionar um aluno a um departamento inexistente.
- **Ações:** `ON DELETE CASCADE` (apagar alunos se o departamento for apagado) ou `ON DELETE SET NULL`.

A base teórica do SQL.

- **Seleção (σ):** Filtrar linhas (SQL WHERE).
- **Projeção (π):** Selecionar colunas (SQL SELECT col1, col2).
- **Junção ($\bowtie$):** Combinar tabelas.
- **União ($\cup$):** Combinar resultados de duas consultas.

O processo de organizar os dados para reduzir a redundância e melhorar a integridade.

- **1NF (Primeira Forma Normal):**

- Cada célula contém um único valor (atômico).
- Sem grupos repetidos de colunas.
- Cada linha é única (tem uma Chave Primária).

- **2NF (Segunda Forma Normal):**

- Está na 1NF.
- Todos os atributos não-chave dependem totalmente de *toda* a chave primária (sem dependências parciais).

- **3NF (Terceira Forma Normal):**

- Está na 2NF.
- Sem dependências transitivas (atributos não-chave não devem depender de outros atributos não-chave).

Objetivo: “The key, the whole key, and nothing but the key, so help me Codd.”

- **Um-para-Um (1:1):**
 - Um utilizador tem um perfil.
 - Raro; geralmente combinados numa única tabela.
- **Um-para-Muitos (1:N):**
 - Um departamento tem muitos alunos.
 - O tipo mais comum.

- **Muitos-para-Muitos (N:M):**
 - Muitos alunos estão inscritos em muitos cursos.
 - **A Solução:** Uma “Tabela de Junção” (ou Tabela Associativa) que armazena pares de IDs.
 - Tabela `inscricoes`: `aluno_id`, `curso_id`.

Vantagens do RDBMS

Um Sistema de Gestão de Bases de Dados Relacionais (RDBMS) garante a integridade dos dados através do ACID:

- **Atomicidade:** “Tudo ou nada.” Se parte de uma transação falhar, tudo é revertido.
- **Consistência:** A base de dados deve seguir sempre as suas regras (restrições).

- **Isolamento:** Transações a acontecer ao mesmo tempo não interferem umas com as outras.
- **Durabilidade:** Uma vez confirmada (committed), uma transação permanece mesmo que a energia falhe.

SQL: Structured Query Language

O que é o SQL?

A linguagem padrão para lidar com Bases de Dados Relacionais.

- É uma linguagem **declarativa**: Diz à base de dados o *que* quer, não *como* obter.
- **Padrão**: ANSI/ISO SQL, embora cada fornecedor (MySQL, PostgreSQL, Oracle) adicione o seu próprio “sabor”.

Categorias de SQL

1. **DDL (Data Definition Language):** Define a estrutura (esquema).
2. **DML (Data Manipulation Language):** Lida com os dados em si.
3. **DQL (Data Query Language):** Lida com consultas (por vezes incluída na DML).
4. **DCL (Data Control Language):** Lida com permissões (GRANT/REVOKE).
5. **TCL (Transaction Control Language):** COMMIT/ROLLBACK.

DDL: Data Definition Language

Antes de criar uma tabela, devemos escolher os tipos corretos:

- **INTEGER:** Números inteiros.
- **TEXT / VARCHAR(N):** Cadeias de texto.
- **REAL / FLOAT:** Números com decimais.
- **BOOLEAN:** Verdadeiro ou Falso.
- **DATE / TIMESTAMP:** Datas e horas.
- **BLOB:** Binary Large Objects (imagens, ficheiros).

Criar uma Tabela

```
CREATE TABLE alunos (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nome TEXT NOT NULL,  
    idade INTEGER CHECK (idade > 0),  
    email TEXT UNIQUE,  
    dept_id INTEGER,  
    data_inscricao DATE DEFAULT CURRENT_DATE,  
    FOREIGN KEY (dept_id) REFERENCES departamentos(id)  
);
```

Restrições (Constraints)

Regras que a base de dados impõe para garantir a qualidade dos dados:

- **NOT NULL:** A coluna não pode estar vazia.
- **UNIQUE:** Todos os valores na coluna devem ser diferentes.
- **PRIMARY KEY:** UNIQUE + NOT NULL + Identificador.
- **FOREIGN KEY:** Liga a outra tabela.
- **CHECK:** Garante que os valores seguem uma regra.

- **Adicionar uma coluna:** `ALTER TABLE alunos ADD COLUMN cidade TEXT;`
- **Renomear uma tabela:** `ALTER TABLE alunos RENAME TO alunos_ativos;`
- **Eliminar uma tabela:** `DROP TABLE alunos;` (Aviso: Isto é permanente!)

DML: Data Manipulation Language

Inserir Dados

```
INSERT INTO alunos (nome, idade, email, dept_id)  
VALUES ('Mario', 30, 'mario@ua.pt', 1);
```

```
-- Múltiplas linhas de uma vez
```

```
INSERT INTO alunos (nome, idade, email)  
VALUES  
    ('Alice', 22, 'alice@ua.pt'),  
    ('Bob', 25, 'bob@ua.pt');
```

- **Atualizar:** UPDATE alunos SET idade = 31 WHERE id = 1;
- **Eliminar:** DELETE FROM alunos WHERE id = 2;
- **Aviso:** Use sempre uma cláusula WHERE, ou irá atualizar/eliminar *tudo*!

DQL: Consultas Básicas

“Obter tudo da tabela de alunos.” `SELECT * FROM alunos;`

“Obter apenas nomes e idades.” `SELECT nome, idade
FROM alunos;`

A cláusula WHERE permite condições complexas:

- **Operadores:** =, <>, <, >, <=, >=.
- **Lógica:** AND, OR, NOT.
- **Conjuntos:** IN (1, 2, 3).
- **Intervalos:** BETWEEN 18 AND 25.
- **Padrões:** LIKE 'Mar%' (Encontra Mario, Maria).

- **Ordenar por idade (mais novos primeiro):** `SELECT * FROM alunos ORDER BY idade ASC;`
- **Ordenar por nome (ordem alfabética inversa):** `SELECT * FROM alunos ORDER BY nome DESC;`
- **Obter apenas os primeiros 3:** `SELECT * FROM alunos LIMIT 3;`

Aliases (Pseudónimos)

Tornar as colunas de saída ou tabelas mais fáceis de ler.

```
SELECT nome AS nome_aluno, idade * 365 AS idade_em_dias  
FROM alunos AS a  
WHERE a.idade > 20;
```

Sumário

- **Modelo Relacional:** Organiza os dados em tabelas com PKs e FKs.
- **Normalização:** 1NF, 2NF, 3NF reduzem a redundância.
- **ACID:** Garante a integridade e fiabilidade dos dados.
- **DDL:** CREATE, ALTER, DROP.
- **DML:** INSERT, UPDATE, DELETE.
- **DQL:** SELECT com WHERE, ORDER BY e LIMIT.